Forkable sandboxes let computers explore alternatives without doing work twice.

Self-organization → self-driving cars
→ self-driving compute.

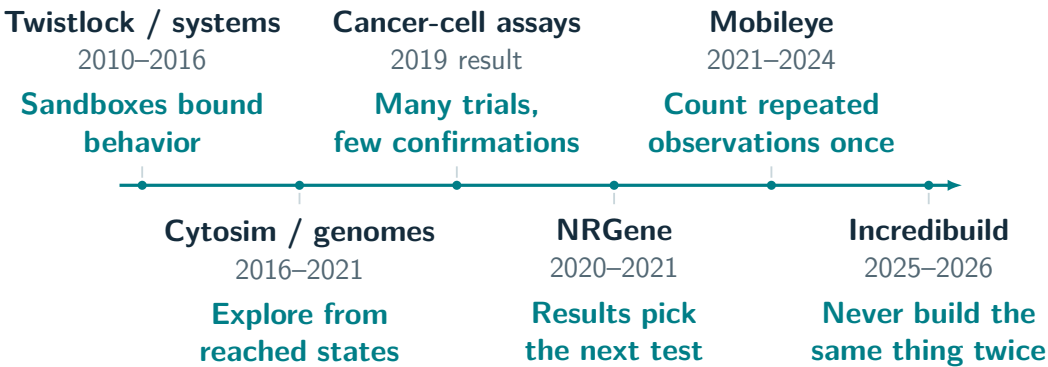


Compute once. Count once. Publish once.

Yossi Eliaz
Principal Engineer, Incredibuild
Associate Professor, HIT

Cambridge Systems Research Group, 15 October 2026. Employer affiliation disclosed.
Tags: PUBLISHED published work MEASURED our measurements BUILT our code
DESIGN proposed, not run ILLUSTRATION worked example

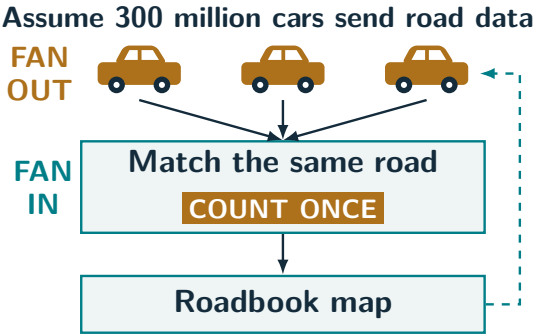
Each stop on my path left one lesson.



Career periods; cancer-drug result (2019). Periods overlap; not to scale. Lessons are my reading, not results.

A car fleet fans out, then fans in to one map.



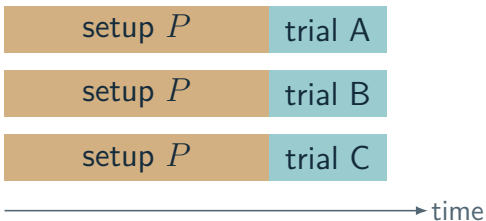


Self-driving compute: many runs from one state, one result.

300M fleet: speaker-reported. Anonymized data flow; Roadbook workflow. Photo: Mobileye test vehicle, Manhattan, June 2021 (press kit). My work there, 2021–2024: junction perception.

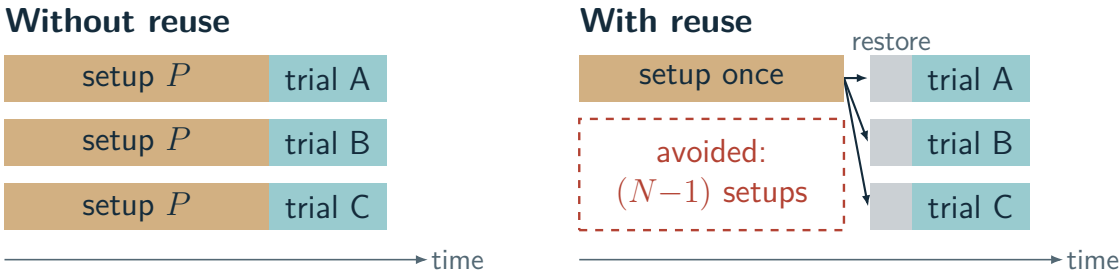
In parallel computing, the costly work is the work done twice.

Without reuse



Schematic, one time scale in both panels. Restoring is not free; the runtime section prices it.

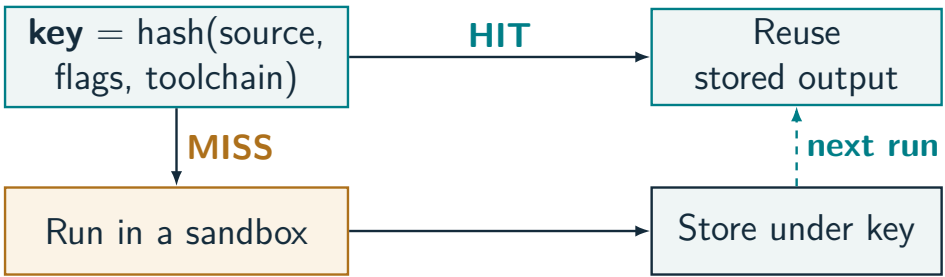
In parallel computing, the costly work is the work done twice.



Three steps remove it: cache, fork, sandbox.

Schematic, one time scale in both panels. Restoring is not free; the runtime section prices it.

A cache skips work a previous run already did.

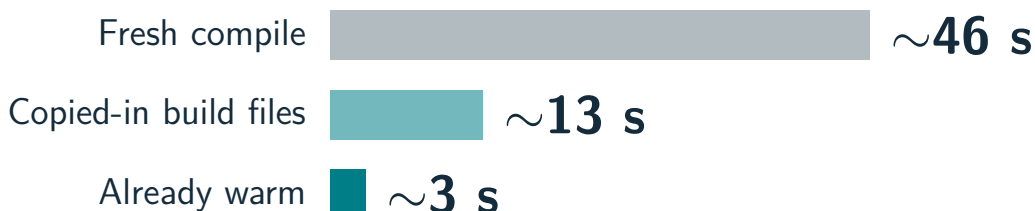


The key must cover every input; the sandbox hides the rest.

As in ccache, Bazel remote caching and Incredibuild. The hash proves identical inputs, not correct outputs.

Reusing an earlier runner's build files cut a Tokio compile from about 46 s to 13 s.

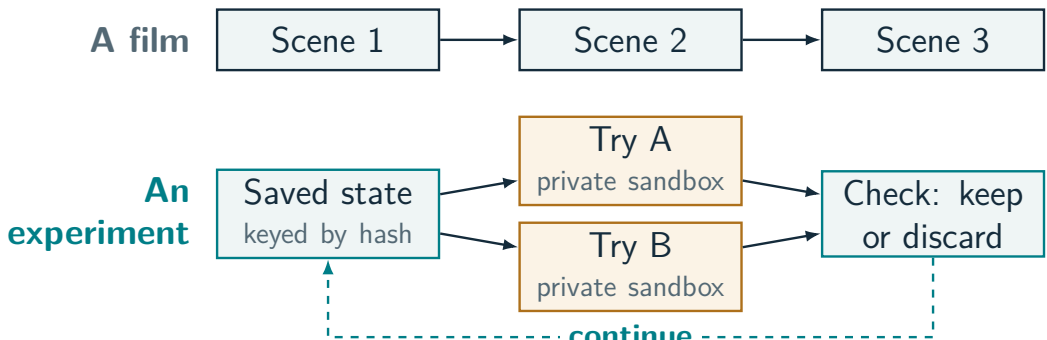
TOKIO COMPILE TIME



Distributed builds rely on it: one machine reuses another's output.

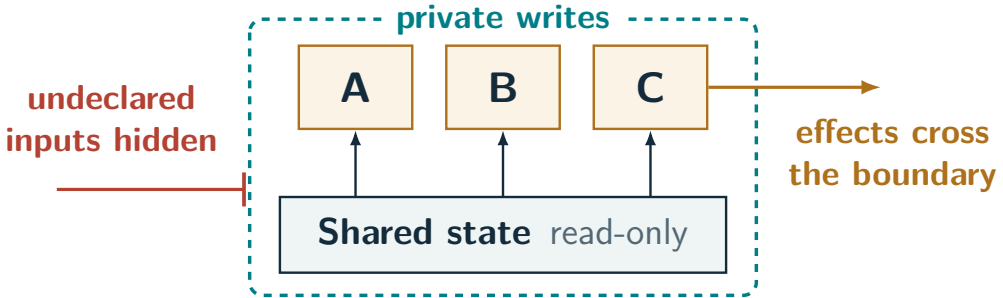
Eliaz, Tokio issue #8200, 8 June 2026: author-reported external Incredibuild demo, approximate compile times.

A fork extends caching from files to a reached execution state.



Organizing analogy and proposed execution model. A restore does not rewind requests already sent.

A sandbox lets children share one state safely.



A sandbox is a policy; a VM or a container is one way to build it.

Diagram: proposed model. Our agent runs used Docker Sandboxes: a microVM with a private Docker daemon.

The forkable API prepares once and runs alternatives in private children.

Python loop

```
out = []
for edit in edits:
    env = prepare()
    env.apply(edit)
    out.append(test(env))
best = select(out)
```

`prepare()` **3** times.

Forkable API

```
parent = checkpoint(prepare())
children = fork(parent, n=3)
execute(children, edits)
out = evaluate(children, test)
best = select(out)
```

`prepare()` **once**; **3** private children.

Proposed API sketch, not an existing SDK. Same three edits, same test, same selection rule in both columns.

Caches, forks and sandboxes are old; coupling them to evidence and acceptance is new.

	Keeps	Examples
Cache	run outputs	ccache, Bazel, Incredibuild
Fork	reached states	SnowFlock (Xen), CRIU, Firecracker
Sandbox	writes private	containers, gVisor, microVMs

New: count shared evidence once; publish checked bytes once.

Not claimed today: a fork speedup (our agent runs were serial) or live-memory restore. SnowFlock: Lagar-Cavilla et al., EuroSys 2009. Firecracker: Agache et al., NSDI 2020.

Sharing work creates three rules.

1 COMPUTE ONCE

Runtime

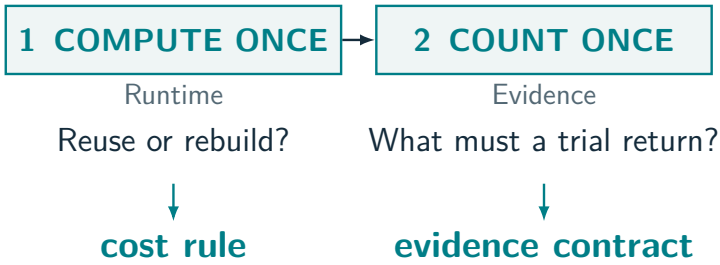
Reuse or rebuild?

↓

cost rule

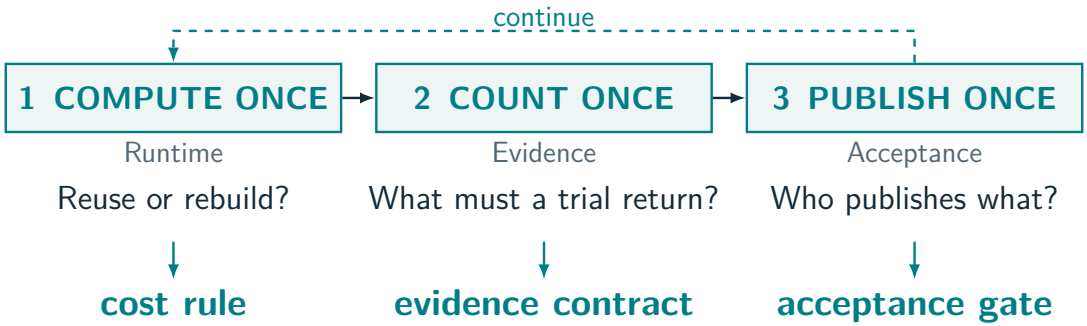
The talk takes the rules in order. This map returns after the examples and at the end.

Sharing work creates three rules.



The talk takes the rules in order. This map returns after the examples and at the end.

Sharing work creates three rules.

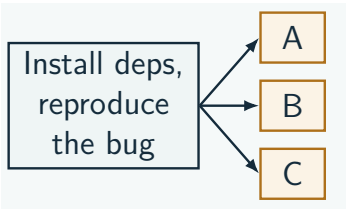


You will leave with all three.

The talk takes the rules in order. This map returns after the examples and at the end.

Example: three agents, one failing test.

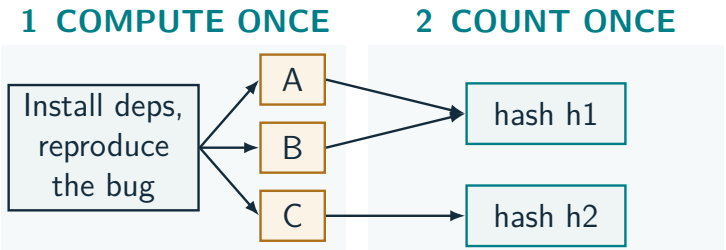
1 COMPUTE ONCE



1 setup, not 3

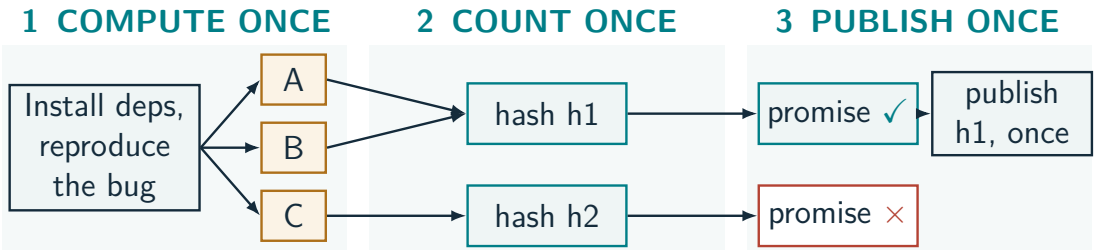
Illustration, not a recorded run: our factory runs were serial, and nothing forked. A hash names a patch's exact bytes.

Example: three agents, one failing test.



Illustration, not a recorded run: our factory runs were serial, and nothing forked. A hash names a patch's exact bytes.

Example: three agents, one failing test.



One setup. Two distinct results. One publish.

Illustration, not a recorded run: our factory runs were serial, and nothing forked. A hash names a patch's exact bytes.

Reuse makes fuzzing fast; only checks turn crashes into findings.

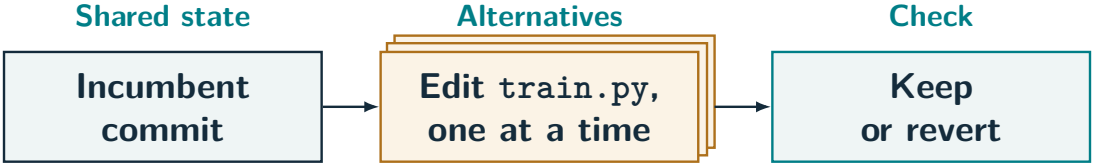


AFL++ **10–20**× faster CyberGym **56** → **22** confirmed

Set up once; count each bug once. COMPUTE ONCE COUNT ONCE

Two separate sources. AFL++ persistent-mode docs: typical gain vs a fork per input. CyberGym report: OpenHands + GPT-5 wrote and ran inputs on 431 projects (latest-code campaign).

Autoresearch kept 23 of 126 attempts against a fixed evaluator.



`val_bpb` **0.9979 → 0.9697** **2.8%** lower

One start, one fixed evaluator. COMPUTE ONCE PUBLISH ONCE

Karpathy, autoresearch protocol; one reported session, 5-minute runs, lower `val_bpb` is better. Same loop over a population of programs: AlphaEvolve (2025), a Borg heuristic recovered 0.7% of Google's compute, on average.

250 samples raised SWE-bench Lite coverage from 15.9% to 56%.



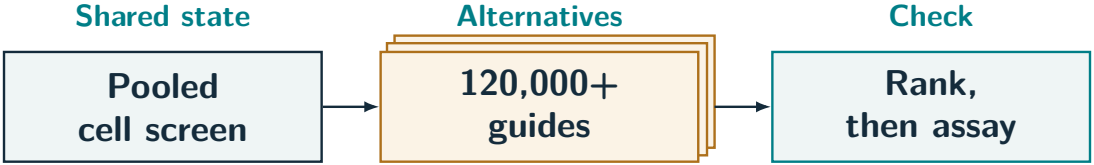
Tasks with a correct patch: **15.9% → 56%**

Coverage is not selection: one patch ships. PUBLISH ONCE

Brown et al., Large Language Monkeys (2024): DeepSeek-Coder-V2-Instruct on SWE-bench Lite; coverage = tasks where at least one sampled patch passes the hidden tests.

PUBLISHED

Our CRISPR screen narrowed 19,050 genes to about 100 candidates.



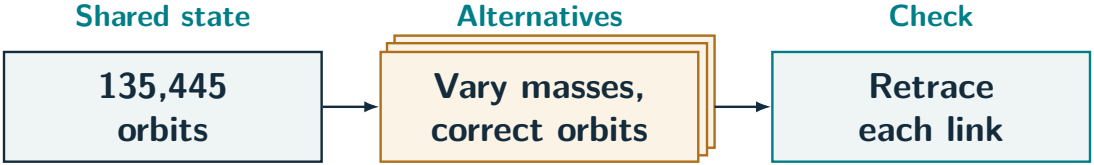
Cell assays: **10 of 13** targets, **6 of 8** combinations

Six guides, one gene.

COUNT ONCE

Gilad, Eliaz et al., Communications Biology 4:399 (2021), Fig. 3: six guides per gene, nine pooled arms; follow-up in MCF-7 cells, in vitro.

With Ori Chamo, we connected two apparent branches of one orbit family.

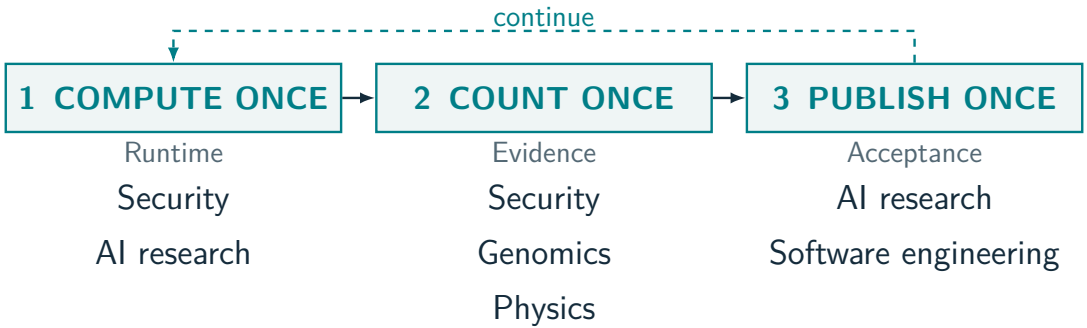


26 of 26 selected connections hold both ways

Two plotted branches, one family. COUNT ONCE

With **Ori Chamo**. Chamo–Eliaz preprint (2026), not peer reviewed; catalog of Li, Li & Liao; Atlas numerical records.

Five fields, one loop, three rules.



Compute once. Count once. Publish once.

The five examples, each under the rule it showed. None measures our integrated runtime.

The next step decides which state to reuse.

Next step	Reuse by	Supplies
Rebuild	**Cache**	files
Continue a live run	**Restore**	its memory
Reuse an answer	**Replay**	the recording

MEASURED Restore fan-out on islo: **255/256** succeeded.

DESIGN Live memory: untested (Gate 0).

Restore fan-out: many restores of one named snapshot (arXiv:2607.09689v4, Table 2). See also CRIU, Firecracker, rr.

A restore pays only if it beats a warm cache.

$$(N-1)P > H + N(R+D)$$

avoided **overhead**

N	runs (8)
P	setup per run
H	save once (2)
$R+D$	restart, private writes (1+3)

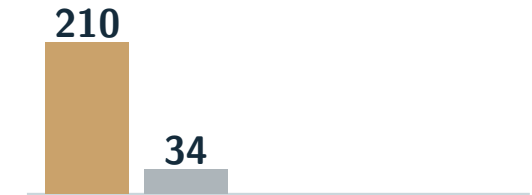
Illustrative model, resource-seconds. Starts: 73 ms median to first command (ComputeSDK Burst TTI, 2 Oct 2026).

A restore pays only if it beats a warm cache.

$$(N-1)P > H + N(R+D)$$

avoided **overhead**

N	runs (8)
P	setup per run
H	save once (2)
$R+D$	restart, private writes (1+3)



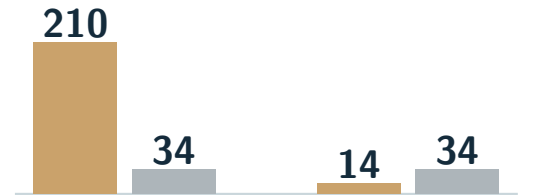
Illustrative model, resource-seconds. Starts: 73 ms median to first command (ComputeSDK Burst TTI, 2 Oct 2026).

A restore pays only if it beats a warm cache.

$$(N-1)P > H + N(R+D)$$

avoided **overhead**

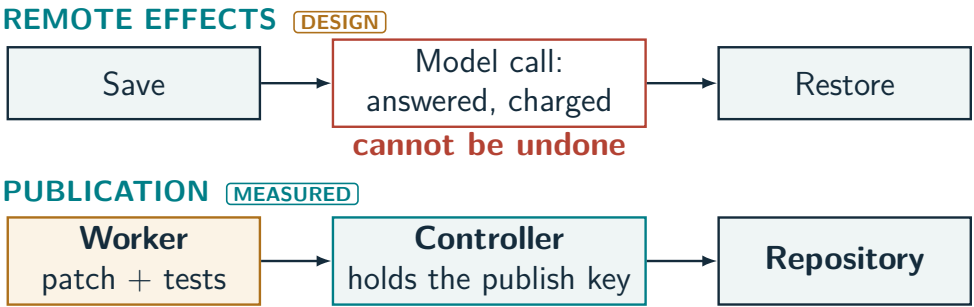
N	runs (8)
P	setup per run
H	save once (2)
$R+D$	restart, private writes (1+3)



Price a restore against a warm cache, not a cold start.

Illustrative model, resource-seconds. Starts: 73 ms median to first command (ComputeSDK Burst TTI, 2 Oct 2026).

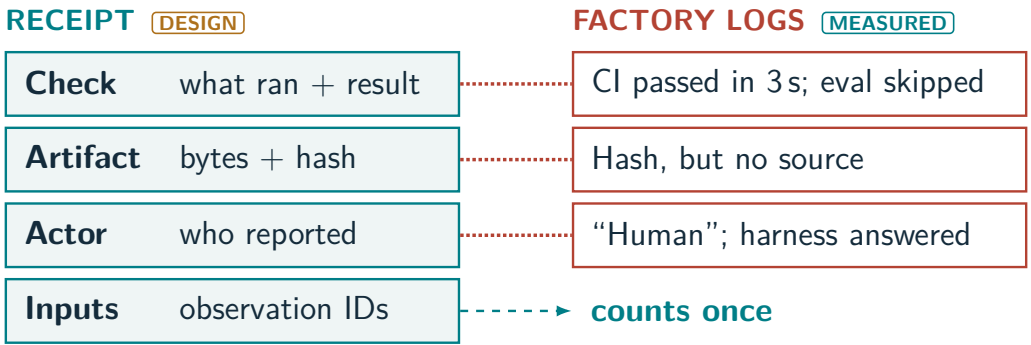
A restore cannot undo remote effects, and a fork must not hold the right to publish.



Record effects once. Keep the publish key outside every fork.

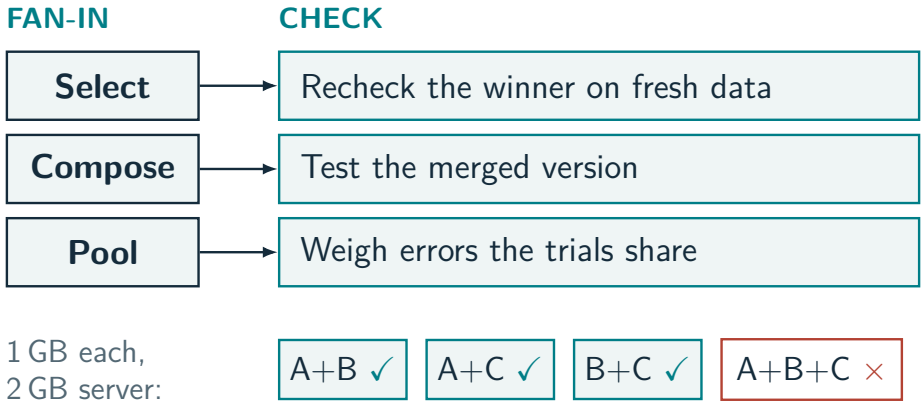
Output commit: Speculator, external synchrony, Remus. Effect journal: proposed. Publish path: our factory runs.

A receipt is evidence's cache entry, written outside the child.



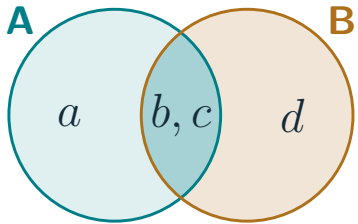
Factory records (SELFHOST-2, 27 Sep; CI, 28 Sep 2026). Receipt fields proposed. Related: PASS, CamFlow, in-toto.

Each fan-in operation needs its own check.



Merge example: illustrative, not a recorded incident. Union and intersection: further fan-in rules (Q&A).

Overlapping trials add less than their count.



4 distinct
2 shared
B adds **1**

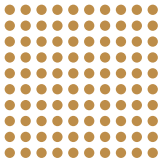
One road, many drives. One gene, six guides.

A shared observation counts once.

BUILT Our reducer rejects duplicate observation IDs.

Illustrative IDs. Callbacks: Roadbook drives, CRISPR guides. Reference reducer; arXiv:2607.09689v4.

At error correlation 0.1, a mean of 100 estimates is as precise as about nine.



$$\longrightarrow N_{\text{eff}} = \frac{100}{1 + 99(0.1)} = \mathbf{9.17} \longrightarrow$$



100 estimates, $\rho = 0.1$

9 independent

Shared errors count for less.

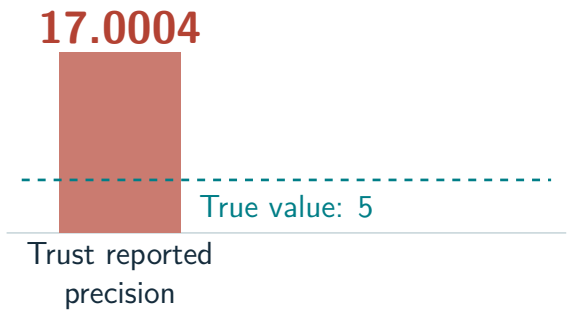
Proposed: covariance-aware weights; refuse unsupported precision claims.

Illustrative: equal noise, common correlation; precision of the mean, not a count of units. arXiv:2607.09689v4.

A worker that overstates its precision can take over the average.

$$\hat{\mu} = \frac{\Sigma_i \, p_i \, x_i}{\Sigma_i \, p_i}$$

p_i = **reported** precision

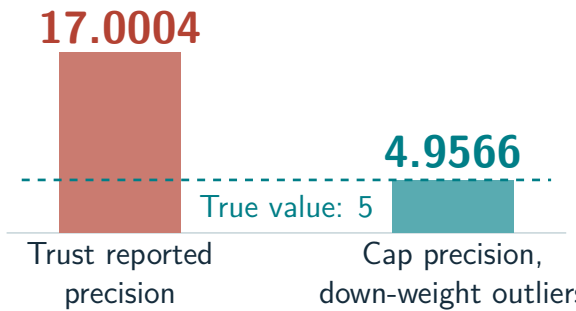


Synthetic test. Reference demo: one stress case, not a Byzantine guarantee; the rule needs trusted precision.

A worker that overstates its precision can take over the average.

$$\hat{\mu} = \frac{\Sigma_i \, p_i \, x_i}{\Sigma_i \, p_i}$$

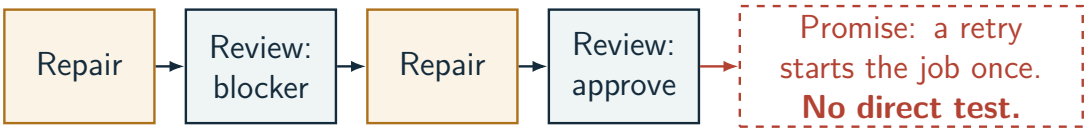
p_i = **reported** precision



Check outside the child, including its error bars.

Synthetic test. Reference demo: one stress case, not a Byzantine guarantee; the rule needs trusted precision.

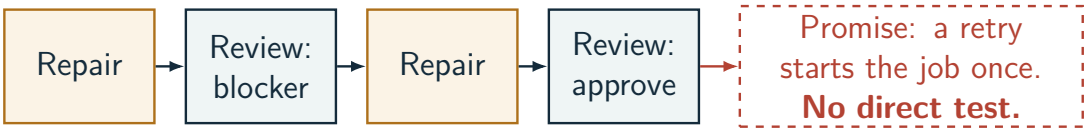
Our factory's patch passed every check it had.



✓ Full suite: **1,972** cases, **0** failures.

SELFHOST-2, 27 Sep 2026: serial, Docker sandboxes, nothing forked; suite run after each repair, cases incl. skips.
PR #2351 (zozo123/ariflow-swfactory) merged 3 Oct 2026 after I reverted the untested path.

Our factory's patch passed every check it had.



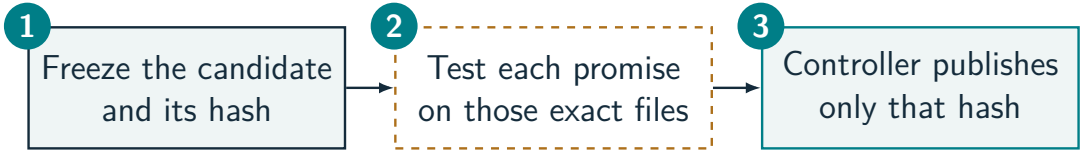
✓ Full suite: **1,972** cases, **0** failures.

Review approved. Suite green.

The new code path had no direct test.

SELFHOST-2, 27 Sep 2026: serial, Docker sandboxes, nothing forked; suite run after each repair, cases incl. skips.
PR #2351 (zozo123/ariflow-swfactory) merged 3 Oct 2026 after I reverted the untested path.

Accept only bytes whose promise was tested.



No test for the promise, no acceptance.

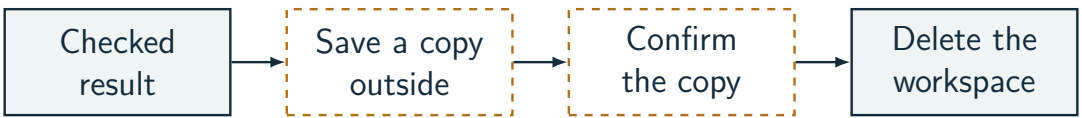
Gate: proposed; the sha256-bound publication check it builds on is built. Promises come from the approved plan and from review findings.

Our second factory run lost an approved patch.

MEASURED

Delivery refused three times; then cleanup **erased the approved patch**.

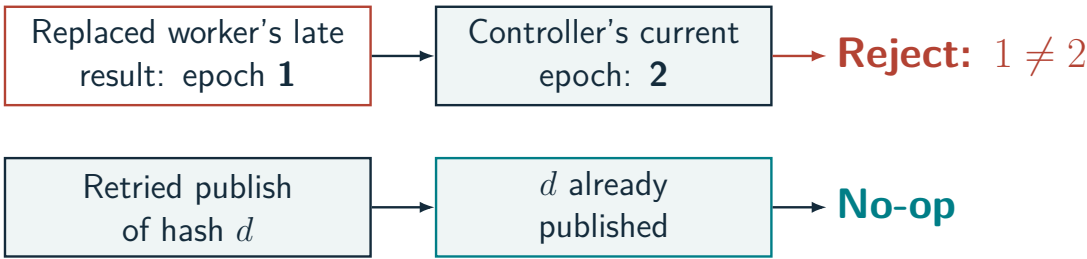
DESIGN



Save outside the child before cleanup.

Second factory run (SELFHOST-3, 27 Sep 2026): serial, Docker sandboxes, nothing forked; local records. Save order: proposed, not executed in the incident.

The controller rejects a late result from a replaced worker.



Proposed: one controller, current epoch, publish once.

Epoch fence and hash-keyed publish: proposed; TLA+ design, no TLC run. Prior art: leases (1989), Chubby sequencers (2006).

Five questions remain open; the preregistered tests have not started.

COMPUTE ONCE

1 Does the snapshot we restore keep live memory? **Gate 0**

2 Does restoring beat a warm cache? **H1**

COUNT ONCE

3 Do restored siblings fail together more? **H2**

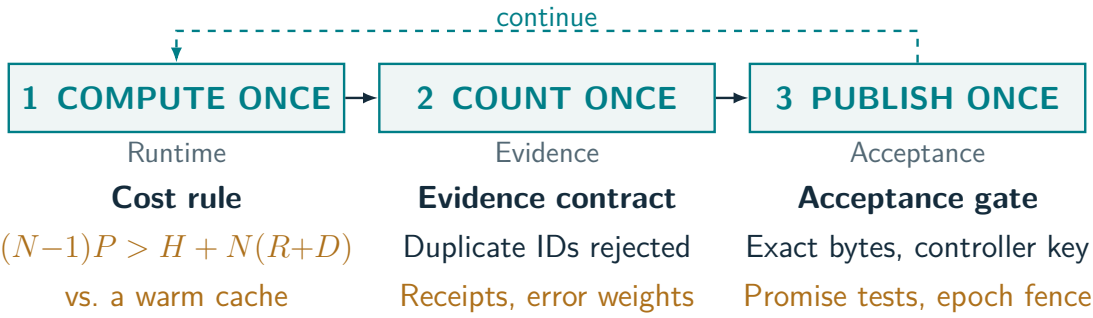
4 When should the reducer refuse a precision claim? **Design**

PUBLISH ONCE

5 Does the epoch fence hold across restores? **Design**

Gate 0, H1, H2: PREREGISTRATION.md, prereg-v1 (amended to v3; no rule changed). Q5: TLA+, no TLC run.

Contributions: three rules for doing work once.



Compute once. Count once. Publish once.

Ink: built (publish check, duplicate-ID reducer). Gold: proposed or modelled. Factory runs: serial, nothing forked.